

Lab 11: Ventus GPGPU Driver Lab

In this lab, you will use the Ventus GPGPU ISA Simulator (based on Spike) and its C++ driver library to study GPGPU kernel programming on RISC-V. You will write vector computation kernels in RISC-V assembly, use the driver API to manage device memory and kernel execution, and run your kernels on the simulator.

By the end of this lab, you will be able to:

1. Build and configure the Ventus GPGPU ISA Simulator and its driver library
2. Write and compile RISC-V assembly kernels with vector instructions for GPU-style execution
3. Use the Ventus driver API to manage the full GPGPU compute pipeline
4. Compare the Ventus GPGPU programming model with CUDA and OpenCL

Default environment configuration:

- **Simulator:** Ventus GPGPU ISA Simulator (Spike-based), cloned from GitHub
- **Toolchain:** RISC-V GNU Toolchain (`riscv64-unknown-elf-*`)
- **Compiler:** GCC 11 required (`gcc-11`, `g++-11`); GCC 13+ may produce build errors
- **Driver library:** `libspike_driver.so`, built from `$SPIKE_SRC_DIR/gpgpu-testcase/driver/`
- **Work directory:** `$SPIKE_SRC_DIR/gpgpu-testcase/`
- **Install target:** `$SPIKE_SRC_DIR/spike/`

Note: After setting environment variables (`SPIKE_SRC_DIR`, `SPIKE_TARGET_DIR`), use the same terminal for the remaining commands or add them to your shell profile.

Step 1 - Environment Setup

Background The Ventus GPGPU ISA Simulator models a RISC-V-based GPU and is built with `autoconf`. The driver library provides a C++ API analogous to CUDA for device memory management and kernel execution. Verify your RISC-V toolchain, install build dependencies, build the simulator, and build the driver.

1.1 Check the RISC-V toolchain and install dependencies:

```
# Verify RISC-V toolchain
riscv64-unknown-elf-as --version
riscv64-unknown-elf-ld --version

# Install build dependency
sudo apt-get install device-tree-compiler

# Verify gcc-11 is available (required for building)
gcc-11 --version
```

1.2 Build the Ventus GPGPU ISA Simulator:

The simulator uses **autoconf** (NOT `cmake`). This is different from some older documentation.

```
git clone https://github.com/THU-DSP-LAB/ventus-gpgpu-isa-simulator --depth=1
cd ventus-gpgpu-isa-simulator
mkdir spike

export SPIKE_SRC_DIR=${PWD}
export SPIKE_TARGET_DIR=${PWD}/spike

mkdir build && cd build
../configure --prefix=${SPIKE_TARGET_DIR} --enable-commitlog
make CC=gcc-11 CXX=g++-11 -j$(nproc)
make install
```

Verify the build:

```
${SPIKE_TARGET_DIR}/bin/spike --help
```

Important: GCC 11 has been verified to work. GCC 13+ may produce errors. If your system defaults to a newer GCC, use `CC=gcc-11 CXX=g++-11` explicitly as shown above.

1.3 Build the driver library:

```
cd ${SPIKE_SRC_DIR}/gpgpu-testcase/driver
cmake -S . -B build
cmake --build build
```

Expected build artifacts:

```
driver/build/libspike_driver.so
driver/build/spike_test
```

1.4 Verify the driver (optional):

```
cd ${SPIKE_SRC_DIR}/gpgpu-testcase/driver/build
cp ../../vsw_testcase/vecadd.riscv .
LD_LIBRARY_PATH=.:${SPIKE_TARGET_DIR}/lib ./spike_test
```

Question 1: Clone the repository, build the simulator and driver following the commands above. Submit a screenshot showing the simulator build ending with `make install` success, the driver build artifacts (`libspike_driver.so`, `spike_test`), and a `whoami` run in the same terminal.

Step 2 - Explore the Ventus Driver Architecture

Background The Ventus driver abstracts GPGPU execution behind a C++ API. Key concepts: device memory allocated at `0x90000000`, kernel code loaded at `0x80000000`, execution controlled via `meta_data` struct, and the kernel must write to `tohost` to signal completion.

AI tools are optional. If used, report the tool, model, prompts, and a brief usefulness/reliability reflection.

Question 2: List the eight steps in the standard Ventus driver API flow (from `vt_dev_open` to `vt_buf_free`). For each step, explain what happens at the hardware/simulator level.

Question 3: The `meta_data` struct contains the following fields: `kernel_size[3]`, `wf_size`, `wg_size`, `metaDataBaseAddr`, `ldsSize`, `pdsSize`, `sgprUsage`, `vgprUsage`. Explain:

- The difference between `wf_size` (threads per warp) and `wg_size` (warps per workgroup), and why `wf_size` determines the vector length ($VLEN = wf_size \times 32$)
- The role of `ldsSize` (local data share) and `pdsSize` (private data share) in GPU memory hierarchy
- Why `metaDataBaseAddr` is set to point to a metadata buffer in device memory along with `pdsBaseAddr`

Source Exploration Record: After Questions 2-3, report how much time you spent on this exploration. If you used AI tools, also attach:

1. Tool name and model used
2. Prompts or questions submitted
3. What the AI tool helped you find
4. What you verified manually in the source code or documentation
5. Any mistakes, limitations, or useful lessons from using the tool

Step 3 - Write and Compile a Vector Kernel

Background Write two RISC-V assembly kernels using RVV instructions:

- **Task A:** Vector addition `C[i] = A[i] + B[i]`
- **Task B:** Vector multiplication `C[i] = A[i] * B[i]`

Both operate on 8-element integer vectors. The kernel must write to `tohost` to signal completion.

Create the following files in your work directory (`$SPIKE_SRC_DIR/gpgpu-testcase/`).

my_link.ld (linker script — used by both kernels):

```
ENTRY(_start)
SECTIONS
{
    /* Kernel loaded at 0x80000000 */
    . = 0x80000000;
    .text : { *(.text) }
    . = ALIGN(0x1000);
}
```

```

.tohost : { *(.tohost) }
.fromhost : { *(.fromhost) }
.data : { *(.data) }
.bss : { *(.bss) }
}

```

Task A: my_vecadd.s — complete every **TODO** section:

```

.section .text
.globl _start

_start:
    # TODO 1: Set up stack pointer (sp = x2) to a valid location
    # Hint: use lui + slli to construct a 32-bit value, e.g., 0x7ff00000
    /* YOUR CODE HERE */

    # Set up addresses (already configured by driver memory allocation)
    lui    x10, 0x900000    # Vector A @ 0x90000000
    addi   x10, x10, 0

    lui    x11, 0x900001    # Vector B @ 0x90000100
    addi   x11, x11, 0

    lui    x12, 0x900002    # Result C @ 0x90000200
    addi   x12, x12, 0

    # TODO 2: Configure vector length for 8 elements of 32-bit (e32, ta,
    ma)
    # Hint: load count into a register, then use vsetvli
    /* YOUR CODE HERE */

    # TODO 3: Load vectors A and B into vector registers
    # Hint: use vle32.v
    /* YOUR CODE HERE */

    # TODO 4: Perform element-wise addition: C = A + B
    # Hint: use vadd.vv
    /* YOUR CODE HERE */

    # TODO 5: Store the result vector to memory at address C
    # Hint: use vse32.v
    /* YOUR CODE HERE */

    # TODO 6: Terminate the kernel
    # Write 1 to the tohost address to signal completion to spike's HTIF
    li     t1, 1
    la     t0, tohost
    sw     t1, 0(t0)
1: j      1b

.section .tohost

```

```

.align 3
tohost:
    .dword 0
fromhost:
    .dword 0

```

Task B: `my_vecmul.s` — modify `my_vecadd.s`. Replace the addition instruction with multiplication. Keep all other code identical:

```

.section .text
.globl _start

_start:
    # (Same stack pointer setup as Task A)
    /* YOUR CODE HERE */

    # (Same address setup as Task A: x10=0x90000000, x11=0x90001000,
x12=0x90002000)
    /* YOUR CODE HERE */

    # (Same vector length config: 8 elements, e32, ta, ma)
    /* YOUR CODE HERE */

    # (Same vector loads for A and B)
    /* YOUR CODE HERE */

    # TODO: Perform element-wise multiplication: C = A * B
    # Hint: use vmul.vv instead of vadd.vv
    /* YOUR CODE HERE */

    # (Same store to C)
    /* YOUR CODE HERE */

    # (Same tohost write exit)
    li    t1, 1
    la    t0, tohost
    sw    t1, 0(t0)
1:  j     1b

.section .tohost
.align 3
tohost:
    .dword 0
fromhost:
    .dword 0

```

Compile the kernels:

```

cd ${SPIKE_SRC_DIR}/gpgpu-testcase
mkdir -p build

# Assemble
riscv64-unknown-elf-as -march=rv64gcv -o build/my_vecadd.o my_vecadd.s
riscv64-unknown-elf-as -march=rv64gcv -o build/my_vecmul.o my_vecmul.s

# Link
riscv64-unknown-elf-ld -T my_link.ld build/my_vecadd.o -o
build/my_vecadd.riscv
riscv64-unknown-elf-ld -T my_link.ld build/my_vecmul.o -o
build/my_vecmul.riscv

```

Verify the compiled binary:

```

riscv64-unknown-elf-objdump -d build/my_vecadd.riscv | grep -E
"vsetvli|vle32|vadd|vse32|tohost"
riscv64-unknown-elf-nm build/my_vecadd.riscv | grep -E "tohost|fromhost"

```

You should see the `tohost` and `fromhost` symbols in the nm output. Without them, spike prints a warning and cannot cleanly exit.

Step 4 - Write a Driver Test Program

Background The driver test program uses the Ventus API. Key functions:

Function	Purpose
<code>vt_dev_open(vt_device_h*)</code>	Initialize the simulator device
<code>vt_buf_alloc(device, size, &vaddr, BUF_TYPE, taskID, kernelID)</code>	Allocate GPU memory
<code>vt_copy_to_dev(device, vaddr, host_ptr, size, taskID, kernelID)</code>	Copy host → device
<code>vt_upload_kernel_file(device, filename, kernelID)</code>	Load <code>.riscv</code> binary
<code>vt_start(device, &meta, taskID)</code>	Launch kernel
<code>vt_copy_from_dev(device, vaddr, host_ptr, size, taskID, kernelID)</code>	Copy device → host
<code>vt_buf_free(device, size, &vaddr, taskID, kernelID)</code>	Release buffer

Note: Call `vt_buf_free` between runs to reset allocations. Avoid `vt_dev_close` (known double-free bug after `vt_start`).

Create **my_test.cpp** in `$SPIKE_SRC_DIR/gpgpu-testcase/`. Complete every **TODO** section:

```
#include "ventus.h"
#include <iostream>
#include <cstring>
#include <thread>
#include <chrono>

using namespace std;

struct meta_data {
    uint64_t kernel_id;
    uint64_t kernel_size[3];
    uint64_t wf_size;
    uint64_t wg_size;
    uint64_t metaDataBaseAddr;
    uint64_t ldsSize;
    uint64_t pdsSize;
    uint64_t sgprUsage;
    uint64_t vgprUsage;
    uint64_t pdsBaseAddr;

    meta_data(uint64_t arg0, uint64_t arg1[], uint64_t arg2, uint64_t arg3,
              uint64_t arg4, uint64_t arg5, uint64_t arg6, uint64_t arg7,
              uint64_t arg8, uint64_t arg9)
        : kernel_id(arg0), wf_size(arg2), wg_size(arg3),
          metaDataBaseAddr(arg4),
            ldsSize(arg5), pdsSize(arg6), sgprUsage(arg7), vgprUsage(arg8),
            pdsBaseAddr(arg9) {
        kernel_size[0] = arg1[0];
        kernel_size[1] = arg1[1];
        kernel_size[2] = arg1[2];
    }
};

// Run a kernel with given device handle
// kernel_type: 0 for vecadd, 1 for vecmul
int run_kernel(vt_device_h p, const int* input_a, const int* input_b,
               int* output_c, int data_size, int kernel_type) {
    // TODO 1: Allocate device memory for inputs and output
    // (Addresses allocated sequentially: A@0x90000000, B@0x90001000,
    C@0x90002000)
    uint64_t vaddr_a, vaddr_b, vaddr_c;
    /* YOUR CODE HERE: vt_buf_alloc */ // for A
    /* YOUR CODE HERE: vt_buf_alloc */ // for B
    /* YOUR CODE HERE: vt_buf_alloc */ // for C

    // TODO 2: Copy input data to device memory
    /* YOUR CODE HERE: vt_copy_to_dev */ // for A
    /* YOUR CODE HERE: vt_copy_to_dev */ // for B

    // TODO 3: Upload the kernel file to the device
    const char* kernel_file = (kernel_type == 0)
```

```

        ? "my_vecadd.riscv"
        : "my_vecmul.riscv";
/* YOUR CODE HERE: vt_upload_kernel_file */

// Configure execution metadata
uint64_t num_workgroups[3] = {1, 1, 1};
// wf_size = 8 → VLEN = 8 * 32 = 256
meta_data meta(0, num_workgroups, data_size, 1,
               0x900000000, 0x1000, 0, 32, 32,
               0x900000000);

// TODO 4: Start kernel execution
/* YOUR CODE HERE: vt_start */

// Wait for kernel to complete (kernel exits via tohost write)
this_thread::sleep_for(chrono::seconds(1));

// TODO 5: Read results back from device to host
/* YOUR CODE HERE: vt_copy_from_dev */ // for C

// Free buffers for next kernel run
vt_buf_free(p, 0, nullptr, 0, 0);

return 0;
}

int main() {
    cout << "=== Ventus GPGPU Driver Lab ===" << endl;

    const int DATA_SIZE = 8;
    int input_a[DATA_SIZE] = {1, 2, 3, 4, 5, 6, 7, 8};
    int input_b[DATA_SIZE] = {2, 3, 4, 5, 6, 7, 8, 9};
    int output[DATA_SIZE] = {0};

    // TODO 0: Open device once (reuse for both kernels)
    vt_device_h p = nullptr;
    /* YOUR CODE HERE: vt_dev_open */

    // ===== Task A: Vector Addition =====
    cout << "\n--- Task A: Vector Addition ---" << endl;
    cout << "A: ";
    for (int i = 0; i < DATA_SIZE; i++) cout << input_a[i] << " ";
    cout << "\nB: ";
    for (int i = 0; i < DATA_SIZE; i++) cout << input_b[i] << " ";
    cout << endl;

    memset(output, 0, sizeof(output));
    run_kernel(p, input_a, input_b, output, DATA_SIZE, 0);

    cout << "C = A + B: ";
    for (int i = 0; i < DATA_SIZE; i++) cout << output[i] << " ";
    cout << endl;

    bool pass_a = true;

```



```

for (int i = 0; i < DATA_SIZE; i++) {
    if (output[i] != input_a[i] + input_b[i]) {
        cout << "  Error at index " << i << ": got " << output[i]
            << ", expected " << input_a[i] + input_b[i] << endl;
        pass_a = false;
    }
}
cout << "Task A: " << (pass_a ? "PASSED" : "FAILED") << endl;

// ===== Task B: Vector Multiplication =====
cout << "\n--- Task B: Vector Multiplication ---" << endl;
cout << "A: ";
for (int i = 0; i < DATA_SIZE; i++) cout << input_a[i] << " ";
cout << "\nB: ";
for (int i = 0; i < DATA_SIZE; i++) cout << input_b[i] << " ";
cout << endl;

memset(output, 0, sizeof(output));
run_kernel(p, input_a, input_b, output, DATA_SIZE, 1);

cout << "C = A * B: ";
for (int i = 0; i < DATA_SIZE; i++) cout << output[i] << " ";
cout << endl;

bool pass_b = true;
for (int i = 0; i < DATA_SIZE; i++) {
    if (output[i] != input_a[i] * input_b[i]) {
        cout << "  Error at index " << i << ": got " << output[i]
            << ", expected " << input_a[i] * input_b[i] << endl;
        pass_b = false;
    }
}
cout << "Task B: " << (pass_b ? "PASSED" : "FAILED") << endl;

cout << "\n=== " << ((pass_a && pass_b) ? "All tests passed!" : "Some
tests FAILED!") << " ===" << endl;

// NOTE: Do NOT call vt_dev_close - spike_device destructor has known
double-free bug
return (pass_a && pass_b) ? 0 : -1;
}

```

Compile the test program:

IMPORTANT: Copy the compiled `.riscv` kernel files into `driver/build/` first. Then compile the test from that directory:

```

# Copy kernels to driver/build/ (same dir as test executable)
cd ${SPIKE_SRC_DIR}/gpgpu-testcase
cp build/my_vecadd.riscv driver/build/
cp build/my_vecmul.riscv driver/build/

```

```
# Compile the test program
cd ${SPIKE_SRC_DIR}/gpgpu-testcase/driver/build

g++ -std=c++17 \
    -I../include -L. \
    -o my_test \
    ../../my_test.cpp \
    -lspike_driver \
    -lpthread
```

Step 5 - Run and Verify

Background The kernel writes to `tohost` to signal completion to spike's HTIF, causing `sim->run()` to return. `timeout` is a safety net for unexpected hangs.

Run the test:

```
cd ${SPIKE_SRC_DIR}/gpgpu-testcase/driver/build

# Set library path (driver lib + spike lib)
LD_LIBRARY_PATH=./:${SPIKE_TARGET_DIR}/lib ./my_test
```

Expected output:

```
=== Ventus GPGPU Driver Lab ===

--- Task A: Vector Addition ---
A: 1 2 3 4 5 6 7 8
B: 2 3 4 5 6 7 8 9
C = A + B: 3 5 7 9 11 13 15 17
Task A: PASSED

--- Task B: Vector Multiplication ---
A: 1 2 3 4 5 6 7 8
B: 2 3 4 5 6 7 8 9
C = A * B: 2 6 12 20 30 42 56 72
Task B: PASSED

=== All tests passed! ===
```

Viewing execution logs (optional):

```
# After running a test, inspect the spike log
cat my_vecadd.riscv.log
```

```
# Check that vector instructions executed correctly
grep -E "vle32|vadd|vmul|vse32" my_vecadd.riscv.log
```

Question 4: Submit your completed `my_test.cpp`, `my_vecadd.s`, `my_vecmul.s` and a screenshot showing the successful execution output. The output must show both Task A and Task B passing with correct results. Immediately after the program finishes, run `whoami` in the same terminal and include that output in the screenshot. Briefly explain which API functions you called and what each one does at the simulator level.

Question 5: In this lab, the kernel writes to `tohost` to signal completion rather than using a normal return instruction. Explain: (a) why a return instruction cannot work in this bare-metal GPGPU execution model, (b) how spike's Host-Target Interface (HTIF) detects and handles the tohost write, and (c) what would happen if the kernel omitted the tohost write.

References

- [Ventus GPGPU ISA Simulator](#)
- [RISC-V Vector Extension Specification](#)
- [CUDA Programming Guide](#)
- [OpenCL Specification](#)